

Guida Esercizi Brevi: Heap, Hash, Huffman, Master Theorem e Induzione

PARTE I - HEAP E CODE DI PRIORITÀ

Definizioni e Rappresentazione

Heap = **Albero binario quasi completo** implementato come array con proprietà:

- **Max-heap:** $A[i] \geq A[\text{Left}(i)]$ e $A[i] \geq A[\text{Right}(i)]$
- **Min-heap:** $A[i] \leq A[\text{Left}(i)]$ e $A[i] \leq A[\text{Right}(i)]$

Navigazione Array:

- Radice: $A[1]$
- Parent(i): $A[\lfloor i/2 \rfloor]$
- Left(i): $A[2i]$
- Right(i): $A[2i+1]$

Operazioni Fondamentali

Max-Heapify (Correttezza $O(\log n)$)

python

```
MAX_HEAPIFY(A, i):
    left = 2*i
    right = 2*i + 1
    largest = i

    if left ≤ A.heap_size and A[left] > A[largest]:
        largest = left
    if right ≤ A.heap_size and A[right] > A[largest]:
        largest = right

    if largest ≠ i:
        exchange A[i] ↔ A[largest]
        MAX_HEAPIFY(A, largest)
```

Dimostrazione Correttezza per Induzione sull'Altezza:

- **Caso base** ($h=0$): Nodo foglia, proprietà mantenuta
- **Passo induttivo:** Se sottoeap sinistro e destro sono corretti, e scambiamo con figlio maggiore, proprietà ripristinata

Build-Max-Heap ($O(n)$ - NON $O(n \log n)$!)

python

```
BUILD_MAX_HEAP(A):  
    A.heap_size = length(A)  
    for i = ⌊length(A)/2⌋ downto 1:  
        MAX_HEAPIFY(A, i)
```

Dimostrazione $O(n)$:

- Maggior parte nodi sono foglie (non richiedono heapify)
- Numero nodi ad altezza h : $\leq \lceil n/2^{(h+1)} \rceil$
- Costo: $\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil n/2^{(h+1)} \rceil \cdot O(h) = O(n)$

Extract-Max e Insert ($O(\log n)$)

python

```
HEAP_EXTRACT_MAX(A):  
    if A.heap_size < 1:  
        error "heap underflow"  
    max = A[1]  
    A[1] = A[A.heap_size]  
    A.heap_size = A.heap_size - 1  
    MAX_HEAPIFY(A, 1)  
    return max  
  
HEAP_INSERT(A, key):  
    A.heap_size = A.heap_size + 1  
    A[A.heap_size] = -∞  
    HEAP_INCREASE_KEY(A, A.heap_size, key)
```

Esempi Tipici Esame

Costruire Max-Heap da [60, 90, 49, 65, 46, 73, 88, 45, 43]:

Processo Build-Max-Heap (partendo da $i = \lfloor 9/2 \rfloor = 4$):

1. $i=4$ (65): Heapify 65,45,43 $\rightarrow$ 65,45,43
2. $i=3$ (49): Heapify 49,73,88 $\rightarrow$ 88,73,49
3. $i=2$ (90): Heapify 90,65,88 $\rightarrow$ 90,65,88
4. $i=1$ (60): Heapify 60,90,88 $\rightarrow$ 90,65,88,60,46,73,49,45,43

Risultato: [90, 65, 88, 60, 46, 73, 49, 45, 43]

PARTE II - TABELLE HASH

Funzioni di Hash Fondamentali

Metodo della Divisione

$$h(k) = k \bmod m$$

Regole di Progettazione:

- m primo, non vicino a potenze di 2
- Evitare $m = 2^p$ (considera solo p bit meno significativi)

Metodo della Moltiplicazione

$$h(k) = \lfloor Lm(kA \bmod 1) \rfloor$$

Valore suggerito: $A = (\sqrt{5}-1)/2 \approx 0.618034$

Gestione Collisioni

Chaining (Concatenamento)

- Ogni slot contiene lista concatenata
- **Fattore di carico:** $\alpha = n/m$
- **Complessità attesa:** $O(1 + \alpha)$

Open Addressing

Linear Probing: $h'(k,i) = (h(k) + i) \bmod m$

- **Problema:** Clustering primario

Quadratic Probing: $h'(k,i) = (h(k) + c_1i + c_2i^2) \bmod m$

- **Requisito:** c_1, c_2 appropriati per garantire permutazione

Double Hashing: $h'(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

- **Requisito:** $h_2(k)$ coprimo con m
- **Migliore distribuzione** tra i metodi

Esempi Tipici Esame

Double Hashing $m=8$, $h_1(k)=k \bmod 8$, $h_2(k)=1+2(k \bmod 6)$

Inserimento [13, 29, 19, 27, 8]:

- $k=13$: $h_1(13)=5$, slot libero $\rightarrow$ posizione 5
- $k=29$: $h_1(29)=5$, occupato, $h_2(29)=1+2(29 \bmod 6)=1+2\cdot 5=11 \bmod 6=5 \rightarrow (5+1\cdot 5) \bmod 8 = 2 \rightarrow$ posizione 2
- $k=19$: $h_1(19)=3$, slot libero $\rightarrow$ posizione 3
- $k=27$: $h_1(27)=3$, occupato, $h_2(27)=1+2(27 \bmod 6)=1+2\cdot 3=7 \rightarrow (3+1\cdot 7) \bmod 8 = 2$, occupato $\rightarrow (3+2\cdot 7) \bmod 8 = 1 \rightarrow$ posizione 1
- $k=8$: $h_1(8)=0$, slot libero $\rightarrow$ posizione 0

Funzione Non Appropriata: $h(k,i) = (k + 2i) \bmod m$

- **Problema:** Se $m=10$, $k=0$ genera sequenza 0,2,4,6,8,0,2,4,6,8...
- **Non permutazione** completa di $\{0,1,\dots,m-1\}$

Hash Uniforme e Proprietà Speciali

Invarianza per Permutazioni

$h(k) = k \bmod m$ con $m = 2^p - 1$ è invariante rispetto a permutazioni delle p -word che costituiscono k .

Dimostrazione per Induzione:

- **Base $k=0$:** Banale
- **Passo $k \rightarrow k+1$:** Usando $n = a_{\{k+1\}}b^{\{k+1\}} + n'$, si dimostra che $n \bmod (b-1) = (a_{\{k+1\}} + n' \bmod (b-1)) \bmod (b-1) = \sum_{j=0}^{k+1} a_j \bmod (b-1)$

PARTE III - CODIFICA DI HUFFMAN

Algoritmo di Costruzione

python

```
HUFFMAN(C):
    n = |C|
    Q = C # Min-heap per frequenze

    for i = 1 to n-1:
        z = allocate_node()
        x = EXTRACT_MIN(Q)
        y = EXTRACT_MIN(Q)
        z.left = x
        z.right = y
        z.freq = x.freq + y.freq
        INSERT(Q, z)

    return EXTRACT_MIN(Q) # Radice dell'albero
```

Esempi Completi dagli Esami

Alfabeto {a,b,c,d,e,f,g} con frequenze [18,32,11,10,4,4,21]

Processo Step-by-Step:

1. Min-heap: [(4,e), (4,f), (10,d), (11,c), (18,a), (21,g), (32,b)]
2. Estrarre e,f → Nuovo nodo (8, e+f)
3. Min-heap: [(8,e+f), (10,d), (11,c), (18,a), (21,g), (32,b)]
4. Estrarre (8,e+f),d → Nuovo nodo (18, (e+f)+d)
5. Continuare fino a radice finale

Codici Risultanti (convenzione: 0=sinistra, 1=destra):

- b: 10, g: 11, a: 00, c: 010, d: 0110, e: 01110, f: 01111

Alfabeto {a,b,c,d,e,f,g} con frequenze [6,21,12,8,3,23,8]

Albero Risultante:



Codici: f:0, b:10, c:110, e:1110, d:1111, g:111, a:11

Proprietà di Ottimalità

Teorema: Il codice di Huffman è ottimo tra tutti i codici prefissi per l'alfabeto dato.

Dimostrazione: Due proprietà greedy:

1. Due caratteri con frequenza minima possono stare alla profondità massima
2. Due caratteri con frequenza minima possono essere fratelli

PARTE IV - MASTER THEOREM E RICORRENZE

Forma Standard

$$T(n) = aT(n/b) + f(n) \text{ dove } a \geq 1, b > 1$$

Tre Casi Fondamentali

Caso 1: $f(n) = O(n^{\log_b(a)-\epsilon})$ per $\epsilon > 0$

Risultato: $T(n) = \Theta(n^{\log_b(a)})$ **Intuizione:** Costo dominato dalle foglie dell'albero di ricorsione

Caso 2: $f(n) = \Theta(n^{\log_b(a)})$

Risultato: $T(n) = \Theta(n^{\log_b(a)} \cdot \log n)$ **Intuizione:** Costo bilanciato tra livelli

Caso 3: $f(n) = \Omega(n^{\log_b(a)+\epsilon})$ per $\epsilon > 0$ E $af(n/b) \leq cf(n)$ per $c < 1$

Risultato: $T(n) = \Theta(f(n))$ **Intuizione:** Costo dominato dalla radice

Esempi Risolti dagli Esami

$T(n) = 4T(n/2) + n \log n$

- $a=4, b=2, f(n)=n \log n$
- $n^{\log_b(a)} = n^{\log_2(4)} = n^2$
- $f(n) = n \log n = O(n^{2-\epsilon})$ per qualsiasi $\epsilon < 1$
- **Caso 1:** $T(n) = \Theta(n^2)$

$T(n) = 4T(n/2) + n^{5/2}$

- $a=4, b=2, f(n)=n^{5/2}$
- $n^{\log_b(a)} = n^2$
- $f(n) = n^{5/2} = \Omega(n^{2+1/2})$
- Condizione regolarità: $4(n/2)^{5/2} = n^{5/2}/\sqrt{2} \leq cn^{5/2}$ per $c=1/\sqrt{2} < 1$
- **Caso 3:** $T(n) = \Theta(n^{5/2})$

$T(n) = T(n-2) + 2n$ (Non Master Theorem)

Metodo di Sostituzione: Ipotesi: $T(n) \leq cn^2$

$$\begin{aligned} T(n) &= T(n-2) + 2n \\ &\leq c(n-2)^2 + 2n \\ &= c(n^2 - 4n + 4) + 2n \\ &= cn^2 - (4c-2)n + 4c \\ &\leq cn^2 \end{aligned}$$

quando $4c - 2 \geq 0$, ovvero $c \geq 1/2$.

Metodi Alternativi

Albero delle Ricorrenze

1. Disegnare albero di chiamate ricorsive
2. Calcolare costo per livello
3. Sommare tutti i livelli
4. Determinare numero di livelli (altezza)

Metodo di Sostituzione

1. **Ipotizzare** forma della soluzione
2. **Dimostrare** per induzione (normale o forte)
3. **Determinare** costanti appropriate

PARTE V - INDUZIONE FORTE

Quando Usare Induzione Forte

Induzione Normale Fallisce quando il passo induttivo richiede informazioni su più valori precedenti, non solo $n-1$.

Principio dell'Induzione Forte

Base: $P(0)$ (o $P(1)$) è vera **Passo:** Se $P(k)$ è vera per tutti $k < n$, allora $P(n)$ è vera **Conclusione:** $P(n)$ è vera per tutti $n \in \mathbb{N}$

Esempi dove Serve Induzione Forte

Correttezza di Algoritmi Divide-et-Impera

MergeSort: Per dimostrare correttezza su array di taglia n , dobbiamo assumere correttezza su sottoarray di taglia $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$.

Teorema: `MergeSort(A,p,r)` ordina correttamente `A[p..r]`

Dimostrazione per induzione forte sulla taglia $n = r-p+1$:

- Base ($n=1$): Singolo elemento già ordinato
- Passo: Assumendo correttezza per tutte le taglie $< n$, le chiamate ricorsive su `A[p..q]` e `A[q+1..r]` producono sottoarray ordinati che Merge combina correttamente

Fibonacci e Ricorrenze Multiple

$$T(n) = T(n-1) + 2T(n-2) + c$$

L'induzione normale non funziona perché dobbiamo ipotizzare forme per $T(n-1)$ e $T(n-2)$.

Con Induzione Forte:

Ipotesi: $T(k) \geq d \cdot 2^k$ per tutti $k < n$

Passo: $T(n) = T(n-1) + 2T(n-2) + c$
 $\geq d \cdot 2^{(n-1)} + 2d \cdot 2^{(n-2)} + c$
 $= d(2^{(n-1)} + 2 \cdot 2^{(n-2)})$
 $= d \cdot 2^n \checkmark$

Alberi Binari e Proprietà Strutturali

Teorema: Un albero binario con n nodi interni ha $n+1$ foglie

Induzione Forte: Assumere vera per tutti gli alberi con meno di n nodi interni, poi considerare albero con n nodi dividendo tra sottoalberi sinistro e destro.

Pattern di Riconoscimento

Usa Induzione Forte quando:

- Ricorrenze multiple: $T(n)$ dipende da $T(n-1)$, $T(n-2)$, ecc.
- Algoritmi divide-et-impera: divisione in sottoproblemi non unitari
- Strutture dati ricorsive: alberi, grafi
- Dimostrazioni su insiemi infiniti: numeri primi, partizioni

Usa Induzione Normale quando:

- Ricorrenza singola: $T(n)$ dipende solo da $T(n-1)$
- Progressione sequenziale chiara
- Proprietà numeriche semplici

STRATEGIA ESAMI

Checklist Rapida Heap

- ☐ Build-Max-Heap: $O(n)$, non $O(n \log n)$
- ☐ Operazioni: Extract/Insert $O(\log n)$
- ☐ Rappresentazione array: $\text{Parent}(i) = \lfloor i/2 \rfloor$

Checklist Rapida Hash

- ☐ Linear probing: clustering primario
- ☐ Double hashing: migliore distribuzione
- ☐ Condizione regolarità Caso 3 Master Theorem

Checklist Rapida Huffman

- ☐ Min-heap per frequenze
- ☐ Ottimalità sempre garantita
- ☐ Costruzione bottom-up

Checklist Rapida Master Theorem

- ☐ Identificare a , b , $f(n)$
- ☐ Calcolare $n^{\log_b(a)}$
- ☐ Confrontare con $f(n)$
- ☐ Caso 3: verificare condizione regolarità

Checklist Induzione

- ☐ Normale fallisce? → Prova forte
- ☐ Definire chiaramente ipotesi induttiva
- ☐ Coprire tutti i casi base necessari
- ☐ Passo induttivo deve usare ipotesi per casi $< n$